Appels Réseau et Asynchronisme avec Redux Thunk

Objectifs

1. Simuler une API asynchrone avec des délais réseau et des erreurs aléatoires.
2. Déplacer la récupération initiale des données vers une action asynchrone Redux (`createAsyncThunk`).
3. Gérer les états de cycle de vie d'une requête (`pending`, `fulfilled`, `rejected`) dans un Slice.
4. Mettre à jour l'interface utilisateur avec des retours visuels (indicateurs de chargement, bannières d'erreur).
5. Prévenir les requêtes multiples indésirables grâce au Hook `useEffect`.

Contexte du Projet : OmniMonitor - Connexion au Serveur Distant

Jusqu'à présent, les stations de l'application **OmniMonitor** étaient codées en dur dans l'état initial de Redux. Il est temps de connecter le tableau de bord à la réalité : les données doivent être récupérées depuis un serveur externe au démarrage de l'application. Vous allez mettre en place toute la plomberie asynchrone nécessaire pour gérer le délai d'attente réseau et les éventuelles pannes du serveur.

Étape 1 : Création du Service de Données (Mock API)

Pour cet exercice, nous n'allons pas déployer un vrai serveur Node.js, mais simuler une API réseau grâce aux Promesses JavaScript.

1. Créez un nouveau fichier `src/api/mockApi.js`.
2. Écrivez et exportez une fonction asynchrone `fetchStationsFromAPI()`.
3. Cette fonction doit :
    - Retourner une Promesse (`new Promise(...)`).
    - Utiliser `setTimeout` pour simuler un délai réseau de 2 secondes.
    - Renvoyer le tableau d'objets des stations (que vous copierez depuis votre ancien `initialState`).
    - *Optionnel :* Ajouter une probabilité de 10% de rejeter la promesse avec une erreur (ex: `throw new Error('Erreur 503 : Serveur Météo Indisponible')`) pour tester votre gestion d'erreur.

Étape 2 : Création de l'Action Asynchrone (Thunk)

Vous allez maintenant configurer Redux pour qu'il puisse appeler cette API.

1. Ouvrez `src/store/stationsSlice.js`.
2. Importez `createAsyncThunk` depuis `@reduxjs/toolkit` et votre fausse API `fetchStationsFromAPI` depuis `../api/mockApi`.
3. Créez et exportez un thunk nommé `fetchStations` :
    - Le préfixe de l'action sera `'stations/fetchStations'`.
    - La fonction de rappel asynchrone exécutera `fetchStationsFromAPI()` et retournera son résultat.

```javascript
export const fetchStations = createAsyncThunk(
  'stations/fetchStations',
  async () => {
    const data = await fetchStationsFromAPI();
    return data;
  }
);
```

Étape 3 : Gestion des États dans le Slice

Il faut modifier votre Slice pour réagir au lancement, au succès ou à l'échec du Thunk.

1. Modifiez l'état initial (`initialState`) de `stationsSlice` pour qu'il ressemble à ceci :
    - `list: []` (tableau vide au démarrage).
    - `status: 'idle'` (chaîne de caractères).
    - `error: null` (stockera le message d'erreur).
2. Ajoutez la propriété `extraReducers: (builder) => { ... }` à la configuration de `createSlice`.
3. Ajoutez les trois cas (cases) correspondants au cycle de vie de `fetchStations` :
    - `.addCase(fetchStations.pending, ...)` : passez le `status` à `'loading'`.
    - `.addCase(fetchStations.fulfilled, ...)` : passez le `status` à `'succeeded'` et injectez `action.payload` dans la `list`.
    - `.addCase(fetchStations.rejected, ...)` : passez le `status` à `'failed'` et stockez `action.error.message` dans `error`.

Étape 4 : Déclenchement de la Requête au Montage

Le Store Redux est prêt, il ne reste plus qu'à déclencher l'action depuis l'interface.

1. Ouvrez votre composant racine de vue (par exemple `App.jsx` ou `DashboardPage.jsx`).
2. Importez `useEffect`, `useDispatch` et `useSelector`.
3. Extrayez l'état de la requête depuis Redux : `const { status, error } = useSelector(state => state.stations)`.
4. Ajoutez un `useEffect` qui s'exécute au montage.
5. À l'intérieur, ajoutez une condition : si `status === 'idle'`, alors déclenchez le Thunk via `dispatch(fetchStations())`. (Cette condition empêche de relancer la requête si l'utilisateur navigue entre les pages de la SPA).

Étape 5 : Rendu Conditionnel Visuel

L'utilisateur doit savoir que le système travaille.

1. Dans le même composant de vue, avant le `return` de l'interface principale, ajoutez des rendus conditionnels anticipés :
    - Si `status === 'loading'`, retournez un div ou un composant affichant : `"Connexion aux satellites en cours..."` (ou un spinner CSS).
    - Si `status === 'failed'`, retournez un panneau d'alerte rouge affichant : `"Alerte Système : " + error` ainsi qu'un bouton permettant de retenter la connexion (qui relancera le dispatch).
2. Vérifiez dans votre navigateur que le délai de 2 secondes affiche bien le message de chargement, puis laisse place à la grille des stations.

Critères de Validation

Avant de valider ce segment, vérifiez que :

- La liste des stations est bien vide au rechargement de la page, puis se remplit automatiquement après 2 secondes.
- L'indicateur de chargement s'affiche systématiquement pendant l'attente réseau.
- Si le serveur renvoie une erreur (à simuler manuellement dans `mockApi.js`), l'interface ne crashe pas mais affiche le message d'erreur prévu.
- Le rechargement des données n'est pas déclenché inutilement à chaque changement de vue (grâce à la vérification `status === 'idle'`).